```
  # process line ...
end
```

These keywords can also be used within blocks. Although you can use them with any block, they make the most sense when the block is being used for iteration:

```
i=0
loop do
  i += 1
  next if i < 3
  print i
  break if i > 4
end
```

produces:

```
345
```

A value may be passed to break or next. A value passed to break is returned as the value of the loop when the break is triggered. A value passed to next is effectively lost, while you can pass a value to next, there's no reason to do so. If a conventional loop doesn't execute a break, its value is nil.

Here's a contrived example:

```
result = while (line = gets)
  break(line) if line =~ /answer/
end

process_answer(result) if result
```

Variable Scope, Loops, and Blocks

The while, until, and for loops are built into the language and do not introduce new scope; previously existing locals can be used in the loop, and any new locals created will be available afterward. Depending on what languages you might be used to, this will either seem normal (Python), very weird (Java), or actually a model of relative clarity (JavaScript).

The scoping rules for blocks (such as those used by loop and each) are different. Normally, the local variables created in these blocks are not accessible outside the block:

```
[1, 2, 3].each do |x|
  y = x + 1
end
[x, y]
```

produces:

```
[x, y]
 ^
prog.rb:4:in `<main>': undefined local variable or method `x' for main:Object
(NameError)
```

However, if at the time the block executes a local variable already exists with the same name as that of a variable in the block, the existing local variable will be used in the block. Its value will therefore be available after the block finishes. As the following example shows, this applies to normal variables in the block but not to the block's parameters:

```
x = "initial value"
```